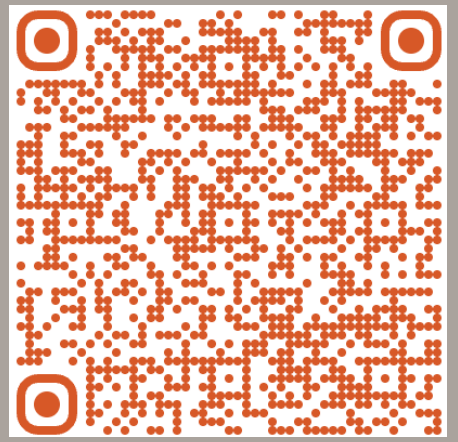
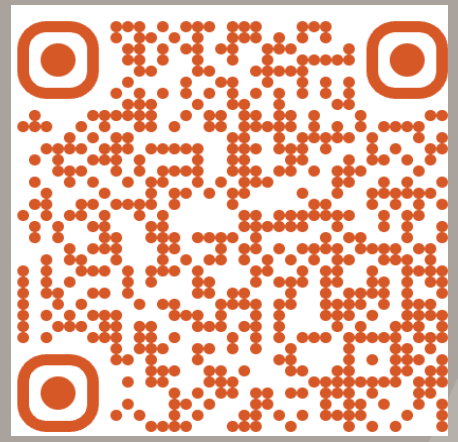
Python for Data Science

Week(5): Files & Input/Output



GitHub Repository



Moodle

Dr. Ghadeer Mobasher



How did you prepare for today's practical lecture? What did you do to prepare? How did you feel after the preparation? For those who missed the last practical lecture, how did you catch up? Were you lost, or was the material clear enough for you to catch up?

Agenda

1. **Files Systems and Directory Structure**

   Hierarchical structure of directories / folders

2. **Reading and Writing Text Files**

   Text File

3. **Working with CSV Files**

   CSV File

4. **Structured Data Formats**

   JSON File

5. **Error Handling and Exception Concepts**

   Error Handling

File Systems and Directory Structure

Concept

- Files are stored in a hierarchical structure of **directories/folders**.
- Python can interact with files and directories using the `os` and `pathlib` modules.

```python
import os
cwd = os.getcwd()
# Print the current working directory to the console
print("Current Directory:", cwd)
# Here, we pass the current working directory (cwd)
# so that Python lists everything inside it.
print("Contents:", os.listdir(cwd))
# os.makedirs() creates a new directory.
os.makedirs("data_folder", exist_ok=True)
# Print a confirmation message after creating the folder
print("New folder created: data_folder")
```

Step 1: Get the current working Directory

It returns the full path of the directory where the current Python script or notebook is running.

Step 2: List all files and folders inside the current directroy

Step 3: Create a new directory. exit_ok = True
→ ignores the request if the folder exists

Exercises

In **Week5.ipynb** , check “**1. File Systems and Directory Structure** ”

In **Week5_Exercise.ipynb** , you have 15 minutes to solve “Problem 1” .

First, download this Colab Notebook
and work via VS or Jupyter Notebook

Don’t forget to split your screen & copy -paste given data / function names.

Reading and Writing Text Files

Writing to a Text File

```
# Writing to a file
with open("example.txt", "w") as f:
    f.write("Hello, students!\n")
    f.write("Welcome to Week 5 - File I/O.\n")
```

Open the file "example.txt" in write mode ('w').
If the file does not exist, it will be created; if it exists, its content will be overwritten.

Reading from a Text File

```
# Reading the file
with open("example.txt", "r") as f:
    content = f.read()
    print(content)
```

Open the file "example.txt" in read mode ('r'), this allows reading the file content without modifying it. Read the entire content of the file as a single string . Print the content to the console.

Notes

- Use `"w"` for writing (overwrites existing file)
- Use `"a"` for appending
- Use `"r"` for reading



What is the difference between "w" and "a" modes?

Update Text File

```
with open('new_reading_file.txt', 'a') as r:
    r.write('We have just created a new file.')
    r.write('\n')
    r.write('A new line has been added.')
    r.write('\n')
# Since we are using with open(),
# the file is automatically closed.
    r.close()

# Open the file again in read mode.
with open('new_reading_file.txt') as r:

# .splitlines() splits the content into separate
# and removes newline characters.
#
# Final result:
# a list of strings
 text = r.read().splitlines()

# ------------------------------------------
# Print the file content
# ------------------------------------------

# Example output:
# ['We have just created a new file.',
#  'A new line has been added.']
print(text)
```

In this example, what if `new_reading_file.txt` doesn't exist?

Write the first line into the file. Add a newline character. Write another line into the file

Read the file and split it into lines. .read() reads the full content as one string. .splitlines () splits the content into separate lines and removes newline characters.

```
['We have just created a new file.','A new line has been added.']
```

Exercises



In **Week5.ipynb**, check "**2. Reading and Writing Text Files**"



In **Week5_Exercise.ipynb**, you have 20 minutes to solve "Problem 2".

Reading and Writing CSV Files

Writing a CSV

```
import csv

data = [
    ["Name", "Age", "Grade"],
    ["Alice", 20, 85],
    ["Bob", 21, 90],
    ["Charlie", 19, 78]
]

with open("students.csv", "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerows(data)
```

Create **a nested list** containing the table data. Each inner list represents one row in the CSV file.

Open a file named "students.csv" in write mode ("w")
newline=“ " prevents extra blank lines in some operating systems.
Create a CSV writer object linked to the file. Write all rows from the 'data' list into the CSV file

Reading a CSV

```
with open("students.csv", "r") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Open the CSV file named "students.csv" in read mode ("r").
Create **a CSV reader object** to read the file content. **Loop through each row in the CSV file. Print the current row as a list**

Creating a CSV file using dictionaries

```python
import csv
# List of dictionaries
data = [
    {'firstname': 'Tom', 'lastname': 'Cruise', 'country': 'USA'},
    {'firstname': 'Tom', 'lastname': 'Hanks', 'country': 'USA'},
    {'firstname': 'Tom', 'lastname': 'Burke', 'country': 'UK'},
    {'firstname': 'Tom', 'lastname': 'Holland', 'country': 'UK'},
]
# Open/Create the CSV file
with open('male_actor_records.csv', 'w', newline='') as csvfile:
# Define the CSV column names
    fieldnames = ['firstname', 'lastname', 'country']
# Create a DictWriter object
    writer = csv.DictWriter(csvfile, fieldnames=fieldnames)

    writer.writeheader()

    writer.writerows(data)

    print("A new file has been created via dictionaries")

    csvfile.close()
```

Defines a **list of dictionaries** called data, where each dictionary represents one actor record with the keys firstname , lastname , and country.

The file male_actor_records.csv is opened in write mode using a **context manager** , which ensures automatic file closure after execution.

A list of column names is defined in fieldnames, specifying the structure and order of the CSV columns.

A **csv.DictWriter** object is then created to map dictionary keys to CSV columns. Thewriteheader () method writes the column names as the first row in the file.

The **writerows (data)** method writes each dictionary in the list as a separate row in the CSV file.

Exercises



In **Week5.ipynb**, check "**3. Working with CSV Files**"



In **Week5_Exercise.ipynb**, you have 25 minutes to solve "Problem 3".

Pause / Break

See you in 15 minutes ☺!
Please be on time..

Reading and Writing JSON Files

Writing JSON

```
import json

student_data = {
    "Alice": {"age": 20, "grade": 85},
    "Bob": {"age": 21, "grade": 90},
    "Charlie": {"age": 19, "grade": 78}
}

with open("students.json", "w") as f:
    json.dump(student_data, f, indent=4)
```

Create a dictionary containing student information
Structure:
key → student name
value → another dictionary containing student details

Write the Python dictionary into the JSON file.
Parameters:
student_data → the data to save
f → the opened file object
indent=4 → formats the JSON with readable indentation

Reading JSON

```
with open("students.json", "r") as f:
    data = json.load(f)
    print(data)
```

Loads and converts the JSON content into a Python dictionary

Access nested JSON data

```python
import json

company = {
    "employee": {
        "name": "David",
        "skills": ["Python", "SQL", "Machine Learning"]
    }
}

print(company["employee"]["name"])

print(company["employee"]["skills"][1])
```



```
David
SQL
```

Update JSON data

```python
import json

user = {
    "username": "admin",
    "active": True
}

user["email"] = "admin@example.com"

user["active"] = False

print(json.dumps(user, indent=4))
```



```
{
    "username": "admin",
    "active": false,
    "email": "admin@example.com"
}
```

Exercises



In **Week5.ipynb** , check “**4. Structured Data Formats (JSON)**”



In **Week5_Exercise.ipynb** , you have 25 minutes to solve “Problem 4” .

Error Handling and Exception Concepts

Concept

- Python can **catch runtime errors** using `try...except`.
- Avoids crashing the program.

Example

```
# Attempt to execute the main program logic
try:
    number = int(input("Enter a number: "))
    print("Reciprocal:", 1/number) # Multiplicative Inverse
except ValueError:
    print("Error: Not a valid number!")
except ZeroDivisionError:
    print("Error: Cannot divide by zero!")
finally:
    print("Execution complete.")
```

Prompt the user to enter a number and convert it to an integer. Compute and print the reciprocal of the number

- Handle the case where input cannot be converted to an integer.
- Handle the case where division by zero occurs.

This block always executes, regardless of whether an exception occurred

Exercises



In Week5.ipynb , check “5. Error Handling and Exception Concepts ”



In **Week5_Exercise.ipynb** , you have 25 minutes to solve “Problem 5” .

Mini Challenge: Class Gradebook

Mini Challenge: Class Gradebook (Command-Line Menu)

You now have every tool you need. Combine them into one small program that a teacher could use.

What the program must do

```
=== Class Gradebook ===
1. Show workspace folders
2. Add a student
3. Show all students
4. Show class average
5. Export JSON report
6. Show activity log
0. Exit
```

Menu option	What it does	Week 5 topic
Start of program	Creates the workspace with `build_workspace(BASE)`	File systems
1	Prints the current directory and the folders inside the workspace	File systems
2	Asks for name, course and grade, then saves the student to `gradebook.csv`	CSV, errors
3	Prints all students in a neat table	CSV
4	Prints the class average and the best student	CSV
5	Saves `reports/report.json` with count, average, best student and all students	JSON
6	Prints every line of `notes/activity_log.txt`	Text files
Every action	Adds one line to the activity log	Text files

Rules

1. Reuse the functions you wrote in Problems 1 to 5.
2. A grade must be a whole number from 0 to 100. Use `safe_int` and show a friendly message for bad input.
3. The program must **never crash**: empty name, text instead of a number, or no students yet must all give a message.
4. To add a student, read the old list, append the new dictionary, and write the full list again.
5. The menu keeps running until the user types `0`.

Already given: the constants, `load_students()`, `show_menu()` and `main()`. You write the seven action functions.

Manual test plan. Run the program and try these steps in order:

1. Choose `3` before adding anyone. You should see a friendly "no students" message.
2. Choose `2` and add Alice, Python, 85.
3. Choose `2` and type `abc` as the grade. You should get an error message, and no student is added.



In **Week5_Exercise.ipynb** , you have 30 minutes to solve “Mini Challenge” .

Coming Up : Week (6)

So far, we have stored data in files and handled errors when reading and writing. How can we now work with data at scale by loading, exploring, cleaning, and summarising datasets using Python's most powerful data tools?

• Role of NumPy in numerical computing • Arrays and vectorized operations • pandas Series and DataFrames • Loading structured datasets• Data selection, filtering, and aggregation
• Basic descriptive statistics • Data cleaning principles

Any Questions?

See you next time ☺

Dr. Ghadeer Mobasher
ghadeer.mobasher@srh-hochschulen.de
https://www.linkedin.com/in/ghadeermobasher/
LGS 6, 2. OG, Arc 213